GM UNIVERSITY

(Established under the Karnataka State Act No. 19 of 2023)
Post Box no-4, PB Road, Davangere-577006
Faculty of Engineering and Technology

School of Computer Science and Technology

LAB MANUAL

Course Name: Artificial intelligence & Machine Learning (AIML)

Course Code: UE24CS2401

Lab Programs

1. Design and implement the A* search algorithm for a given weighted graph. The algorithm should compute the shortest path from a start node to a goal node using: g(n): Cost from start node to current node h(n): Heuristic estimate from current node to goal f(n) = g(n) + h(n).
2. Develop a program to create histograms for all numerical features and analyze the distribution of each feature. Use California Housing dataset.
3. Develop a program to implement Principal Component Analysis (PCA) for reducing the dimensionality of the Iris dataset from 4 features to 2.
4. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Find-S algorithm to output a description of the set of all hypotheses consistent with the training examples.
5. Develop a program to implement k-Nearest Neighbour algorithm to classify the randomly generated 100 values of x in the range of [0,1]. Perform the following based on dataset generated. a) Label the first 50 points {x1,……,x50} as follows: if (xi ≤ 0.5), then xi ∈ Class1, else xi ∈ Class1 b) Classify the remaining points, x51,……,x100 using KNN. Perform this for k=1,2,3,4,5,20,30
6. Develop a program to demonstrate the working of Linear Regression. Use Boston Housing Dataset for Linear Regression.
7. Develop a program to demonstrate the working of the decision tree algorithm. Use Breast Cancer Data set for building the decision tree and apply this knowledge to classify a new sample.
8. Develop a program to implement k-means clustering using Wisconsin Breast Cancer data set and visualize the clustering result.

GM UNIVERSITY

(Established under the Karnataka State Act No. 19 of 2023)
Post Box no-4, PB Road, Davangere-577006

Faculty of Engineering and Technology

School of Computer Science and Technology

Prerequisites for Lab Execution

Step 1: Install Python

- Open browser and visit: https://www.python.org/downloads/
- Download the latest **Python 3.x** version.
- Run the installer.
- ✅ Select "Add Python to PATH"**.** Click Install Now**.**
- Verify installation:
  - Open Command Prompt and Type: python –version
  - Python version should be displayed.

Step 2: Install PyCharm IDE

- **Install Python**
- **Open browser and visit:** https://www.jetbrains.com/pycharm/download/
- Download: **PyCharm Community Edition (Free)** ✅
- Run the downloaded installer.
- Select:
  - Create Desktop Shortcut
  - Add launchers to PATH (optional)
- Click **Install → Finish**.
- Launch PyCharm.

Video Link : https://youtu.be/qaT-2ipbpBg?si=QHxovvoTGwRYSvrk

1. **Design and implement the A* search algorithm for a given weighted graph. The algorithm should compute the shortest path from a start node to a goal node using: g(n): Cost from start node to current node h(n): Heuristic estimate from current node to goal f(n) = g(n) + h(n).**

Program:

```
def aStarAlgo(start_node, stop_node):
    open_set = set(start_node)
    closed_set = set()
    g = {}              #store distance from starting node
    parents = {}        # parents contains an adjacency map of all nodes
    #distance of starting node from itself is zero
    g[start_node] = 0
    #start_node is root node i.e it has no parent nodes
    #so start_node is set to its own parent node
    parents[start_node] = start_node
    while len(open_set) > 0:
        n = None
        #node with lowest f() is found
        for v in open_set:
            if n == None or g[v] + heuristic(v) < g[n] + heuristic(n):
                n = v
        if n == stop_node or Graph_nodes[n] == None:
            pass
        else:
            for (m, weight) in get_neighbors(n):
                #nodes 'm' not in first and last set are added to first
                #n is set its parent
                if m not in open_set and m not in closed_set:
                    open_set.add(m)
                    parents[m] = n
                    g[m] = g[n] + weight
                #for each node m,compare its distance from start i.e g(m) to the
                #from start through n node
                else:
```

```
            if g[m] > g[n] + weight:
                #update g(m)
                g[m] = g[n] + weight
                #change parent of m to n
                parents[m] = n
                #if m in closed set,remove and add to open
                if m in closed_set:
                    closed_set.remove(m)
                    open_set.add(m)
        if n == None:
            print('Path does not exist!')
            return None

        # if the current node is the stop_node
        # then we begin reconstructin the path from it to the start_node
        if n == stop_node:
            path = []
            while parents[n] != n:
                path.append(n)
                n = parents[n]
            path.append(start_node)
            path.reverse()
            print('Path found: {}'.format(path))
            return path
        # remove n from the open_list, and add it to closed_list
        # because all of his neighbors were inspected
        open_set.remove(n)
        closed_set.add(n)
    print('Path does not exist!')
    return None

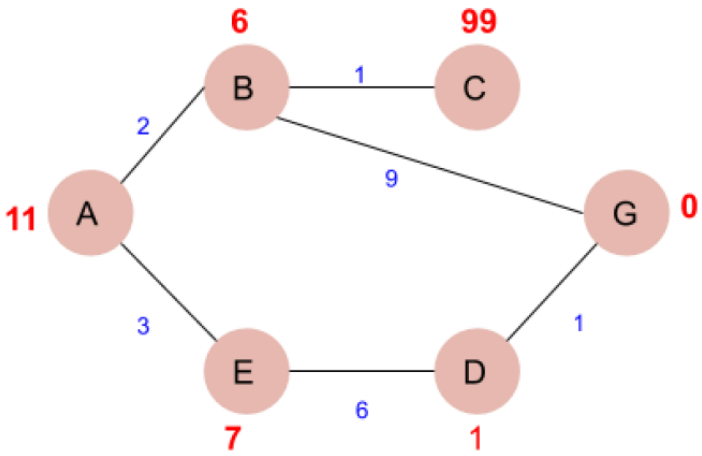
#define fuction to return neighbor and its distance
#from the passed node
```

```
def get_neighbors(v):
    if v in Graph_nodes:
        return Graph_nodes[v]
    else:
        return None
def heuristic(n):
    H_dist = {
        'A': 11,
        'B': 6,
        'C': 99,
        'D': 1,
        'E': 7,
        'G': 0,
    }
    return H_dist[n]

#Describe your graph here
Graph_nodes = {
    'A': [('B', 2), ('E', 3)],
    'B': [('A', 2), ('C', 1), ('G', 9)],
    'C': [('B', 1)],
    'D': [('E', 6), ('G', 1)],
    'E': [('A', 3), ('D', 6)],
    'G': [('B', 9), ('D', 1)]
}

aStarAlgo('A', 'G')
```

Output :



Path found: ['A', 'E', 'D', 'G']

2. Develop a program to create histograms for all numerical features and analyze the distribution of each feature. Use California Housing dataset.

Program :

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.datasets import fetch_california_housing

california_housing = fetch_california_housing()

X = california_housing.data

y = california_housing.target

df = pd.DataFrame(X, columns=california_housing.feature_names)

df["Target"] = y

print("\n📊 Statistical Summary of the Dataset:")

print(df.describe())
```

```
df.hist(bins=30, figsize=(15, 10), color='skyblue', edgecolor='black')

plt.suptitle('Histograms of Numerical Features', fontsize=16)

plt.tight_layout()

plt.show()

plt.figure(figsize=(15, 8))

sns.boxplot(data=df, orient="h", palette="Set2")

plt.title('Box Plots for Numerical Features (Detecting Outliers)', fontsize=16)

plt.tight_layout()

plt.show()


def find_outliers_iqr(data):

    outlier_dict = {}

    for column in data.columns:

            Q1 = data[column].quantile(0.25)

            Q3 = data[column].quantile(0.75)

            IQR = Q3 - Q1

            lower_bound = Q1 - 1.5 * IQR

            upper_bound = Q3 + 1.5 * IQR

            outliers = data[(data[column] < lower_bound) | (data[column] > upper_bound)][column]

            if not outliers.empty:

                    outlier_dict[column] = outliers.values

    return outlier_dict

outliers = find_outliers_iqr(df)
```

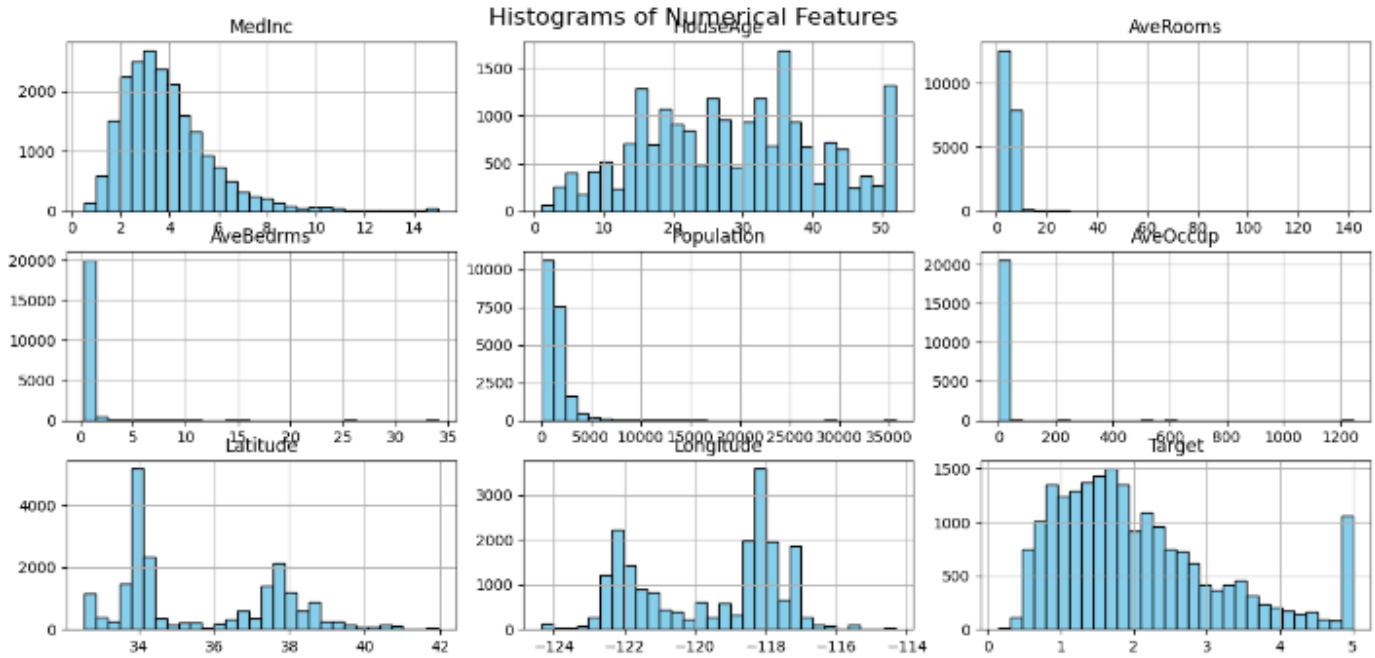
GM UNIVERSITY

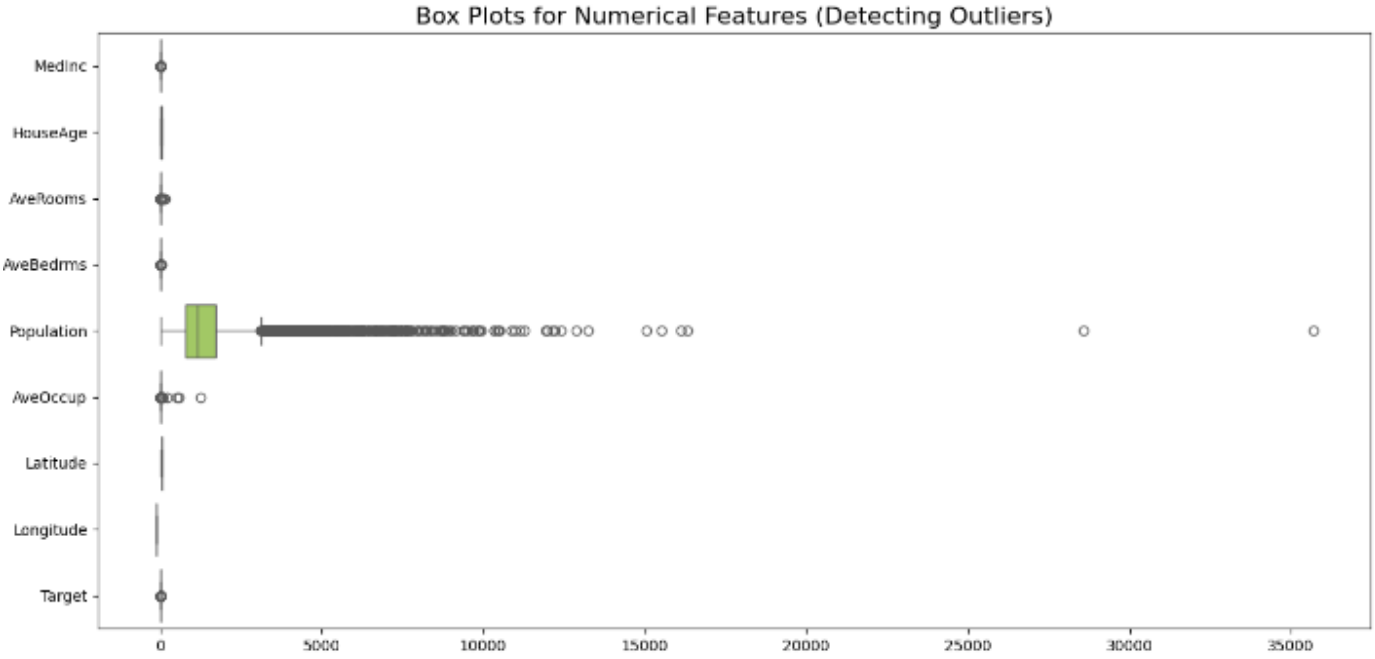
(Established under the Karnataka State Act No. 19 of 2023)
Post Box no-4, PB Road, Davangere-577006

Faculty of Engineering and Technology

School of Computer Science and Technology

Output :





3. Develop a program to implement Principal Component Analysis (PCA) for reducing the dimensionality of the Iris dataset from 4 features to 2.

```
import numpy as np

import pandas as pd

from sklearn.datasets import load_iris

from sklearn.decomposition import PCA

import matplotlib.pyplot as plt

# Load the Iris dataset

iris = load_iris()

data = iris.data

labels = iris.target

label_names = iris.target_names

# Convert to a DataFrame for better visualization

iris_df = pd.DataFrame(data, columns=iris.feature_names)

# Perform PCA to reduce dimensionality to 2

pca = PCA(n_components=2)

data_reduced = pca.fit_transform(data)

# Create a DataFrame for the reduced data

reduced_df = pd.DataFrame(data_reduced, columns=['Principal Component 1', 'Principal Component 2'])

reduced_df['Label'] = labels
```

```
# Plot the reduced data

plt.figure(figsize=(8, 6))

colors = ['r', 'g', 'b']

for i, label in enumerate(np.unique(labels)):

    plt.scatter(

        reduced_df[reduced_df['Label'] == label]['Principal Component 1'],

        reduced_df[reduced_df['Label'] == label]['Principal Component 2'],

        label=label_names[label],

        color=colors[i]

    )

plt.title('PCA on Iris Dataset')

plt.xlabel('Principal Component 1')

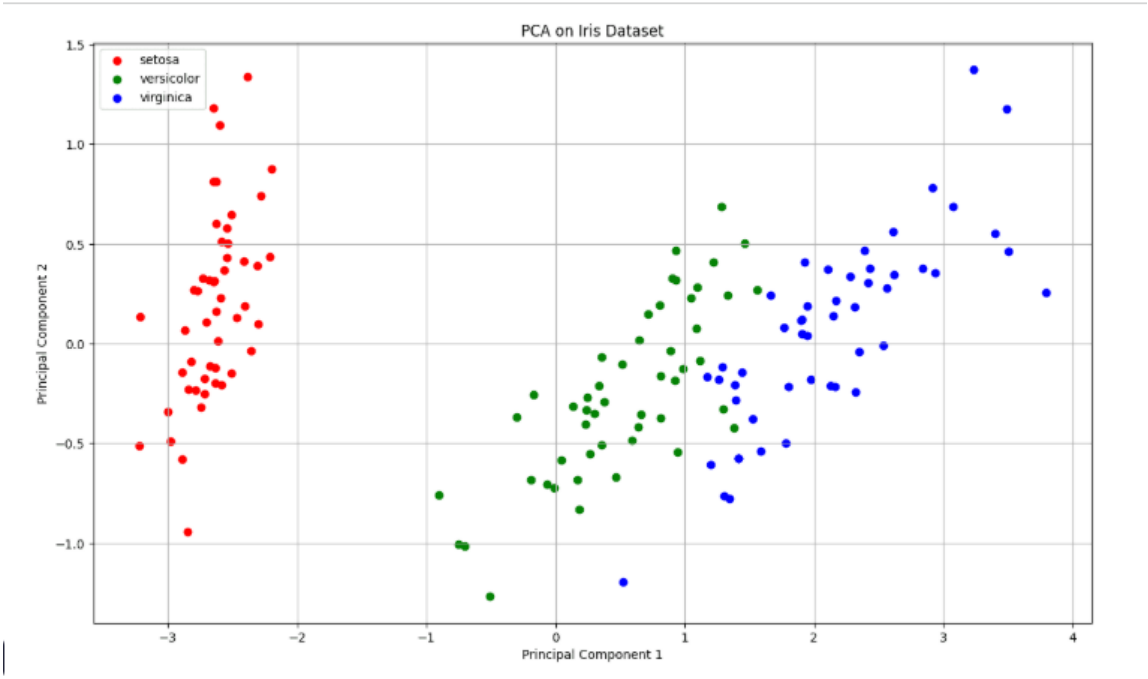
plt.ylabel('Principal Component 2')

plt.legend()

plt.grid()

plt.show()
```

Output:



4. **For a given set of training data examples stored in a .CSV file, implement and demonstrate the Find-S algorithm to output a description of the set of all hypotheses consistent with the training examples.**

```
import pandas as pd
def find_s_algorithm(file_path):
    data = pd.read_csv(file_path)

    print("Training data:")
    print(data)

    attributes = data.columns[:-1]
    class_label = data.columns[-1]

    hypothesis = ['?' for _ in attributes]

    for index, row in data.iterrows():
        if row[class_label] == 'Yes':
            for i, value in enumerate(row[attributes]):
```

```
        if hypothesis[i] == '?' or hypothesis[i] == value:
            hypothesis[i] = value
        else:
            hypothesis[i] = '?'

    return hypothesis
file_path = 'training_data.csv'
hypothesis = find_s_algorithm(file_path)
print("\nThe final hypothesis is:", hypothesis)
```

Output: Excel sheet should include following data and should be named as 'training_data.csv'

Training data:

	Outlook	Temperature	Humidity	Windy	PlayTennis
0	Sunny	Hot	High	False	No
1	Sunny	Hot	High	True	No
2	Overcast	Hot	High	False	Yes
3	Rain	Cold	High	False	Yes
4	Rain	Cold	High	True	No
5	Overcast	Hot	High	True	Yes
6	Sunny	Hot	High	False	No

The final hypothesis is: ['Overcast', 'Hot', 'High', '?']

5. Develop a program to implement k-Nearest Neighbour algorithm to classify the randomly generated 100 values of x in the range of [0,1]. Perform the following based on dataset generated.

a) Label the first 50 points {x1,……,x50} as follows: if (xi ≤ 0.5), then xi ∈ Class1, else xi ∈ Class1

b) Classify the remaining points, x51,……,x100 using KNN. Perform this for k=1,2,3,4,5,20,30

Program:

```
import numpy as np

import matplotlib.pyplot as plt

from collections import Counter

data = np.random.rand(100)

labels = ["Class1" if x <= 0.5 else "Class2" for x in data[:50]]

def euclidean_distance(x1, x2):

    return abs(x1 - x2)

def knn_classifier(train_data, train_labels, test_point, k):

    distances = [(euclidean_distance(test_point, train_data[i]), train_labels[i]) for i in range(len(train_data))]

    distances.sort(key=lambda x: x[0])

    k_nearest_neighbors = distances[:k]

    k_nearest_labels = [label for _, label in k_nearest_neighbors]

    return Counter(k_nearest_labels).most_common(1)[0][0]

train_data = data[:50]

train_labels = labels

test_data = data[50:]



k_values = [1, 2, 3, 4, 5, 20, 30]
```

```
print("--- k-Nearest Neighbors Classification ---")

print("Training dataset: First 50 points labeled based on the rule (x <= 0.5 -> Class1, x > 0.5 -> Class2)")

print("Testing dataset: Remaining 50 points to be classified\n")

results = {}

for k in k_values:

    print(f"Results for k = {k}:")

    classified_labels = [knn_classifier(train_data, train_labels, test_point, k) for test_point in test_data]

    results[k] = classified_labels

    for i, label in enumerate(classified_labels, start=51):

        print(f"Point x{i} (value: {test_data[i - 51]:.4f}) is classified as {label}")

    print("\n")

print("Classification complete.\n")

for k in k_values:

    classified_labels = results[k]

    class1_points = [test_data[i] for i in range(len(test_data)) if classified_labels[i] == "Class1"]

    class2_points = [test_data[i] for i in range(len(test_data)) if classified_labels[i] == "Class2"]

    plt.figure(figsize=(10, 6))

    plt.scatter(train_data, [0] * len(train_data), c=["blue" if label == "Class1" else "red" for label in
train_labels],

                label="Training Data", marker="o")

    plt.scatter(class1_points, [1] * len(class1_points), c="blue", label="Class1 (Test)", marker="x")

    plt.scatter(class2_points, [1] * len(class2_points), c="red", label="Class2 (Test)", marker="x")

    plt.title(f"k-NN Classification Results for k = {k}")
```

```
plt.xlabel("Data Points")

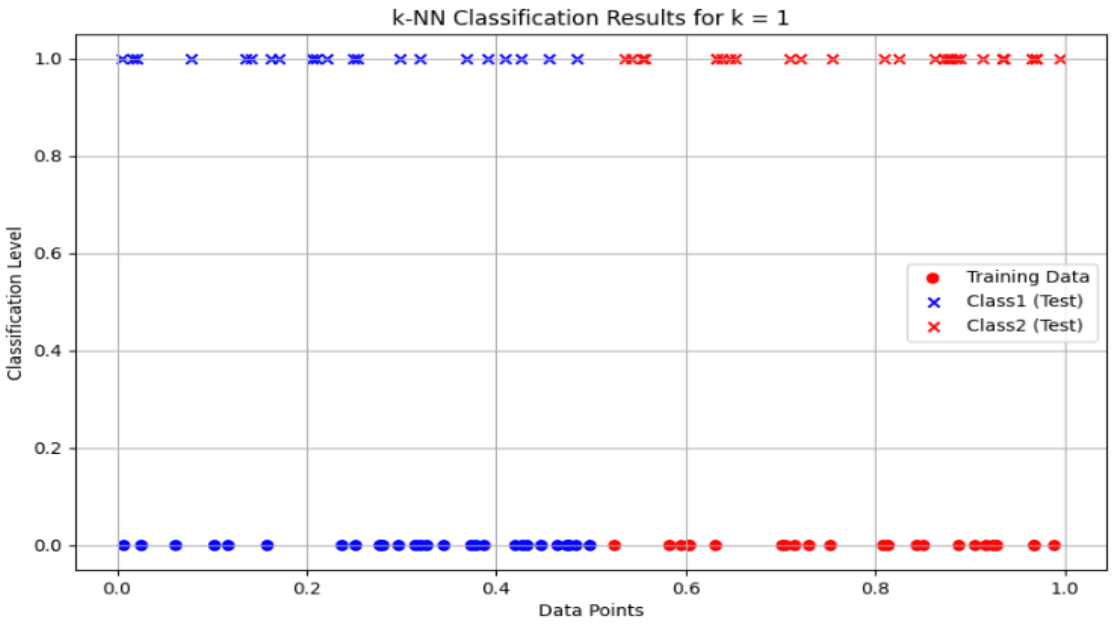
plt.ylabel("Classification Level")

plt.legend()

plt.grid(True)

plt.show()
```

Output :



6. Develop a program to demonstrate the working of Linear Regression. Use Boston Housing Dataset for Linear Regression

Program:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score

def linear_regression_california():
    housing = fetch_california_housing(as_frame=True)
    X = housing.data[["AveRooms"]]
    y = housing.target

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    model = LinearRegression()
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

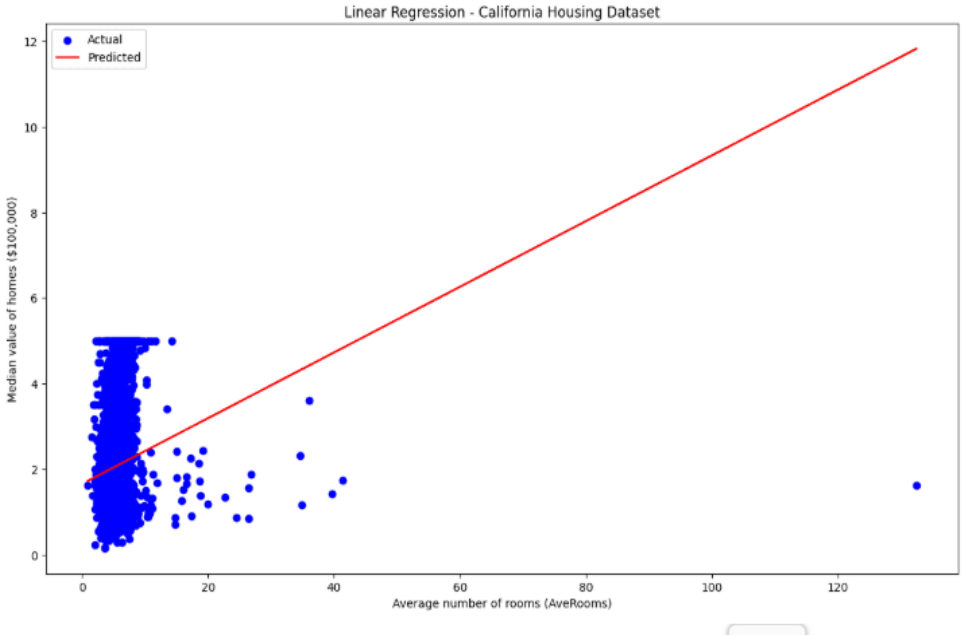
    plt.scatter(X_test, y_test, color="blue", label="Actual")
    plt.plot(X_test, y_pred, color="red", label="Predicted")
    plt.xlabel("Average number of rooms (AveRooms)")
    plt.ylabel("Median value of homes ($100,000)")
    plt.title("Linear Regression - California Housing Dataset")
    plt.legend()
```

```
    plt.show()

    print("Linear Regression - California Housing Dataset")
    print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
    print("R^2 Score:", r2_score(y_test, y_pred))

if __name__ == "__main__":
    print("Demonstrating Linear Regression \n")
    linear_regression_california()
```

Output:



Demonstrating Linear Regression

Linear Regression - California Housing Dataset

Mean Squared Error: 1.2923314440807299

R^2 Score: 0.013795337532284901

7. **Develop a program to demonstrate the working of the decision tree algorithm. Use Breast Cancer Data set for building the decision tree and apply this knowledge to classify a new sample.**

```
# Importing necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn import tree

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")
new_sample = np.array([X_test[0]])
prediction = clf.predict(new_sample)

prediction_class = "Benign" if prediction == 1 else "Malignant"
print(f"Predicted Class for the new sample: {prediction_class}")

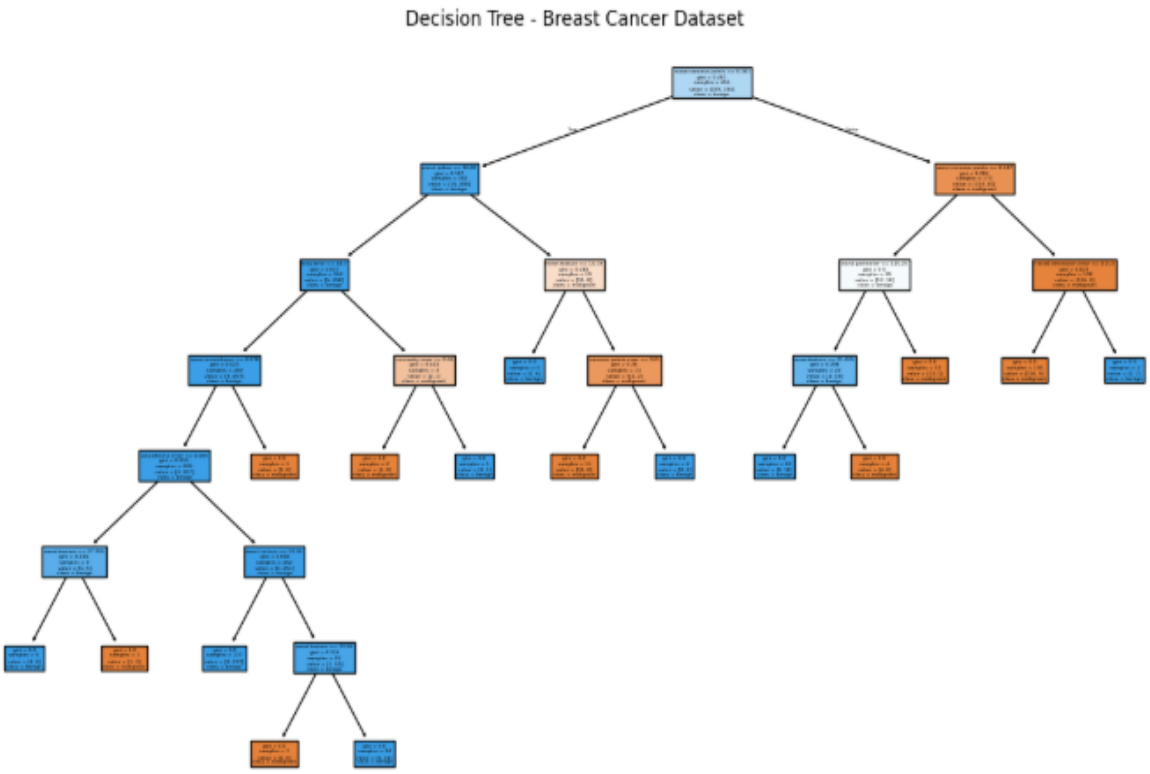
plt.figure(figsize=(12,8))
tree.plot_tree(clf, filled=True, feature_names=data.feature_names,
class_names=data.target_names)
```

```
plt.title("Decision Tree - Breast Cancer Dataset")
plt.show()
```

Output:

Model Accuracy: 94.74%

Predicted Class for the new sample: Benign



8. **Develop a program to implement k-means clustering using Wisconsin Breast Cancer data set and visualize the clustering result.**

Program

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import confusion_matrix, classification_report

data = load_breast_cancer()
X = data.data
y = data.target

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=2, random_state=42)
y_kmeans = kmeans.fit_predict(X_scaled)

print("Confusion Matrix:")
print(confusion_matrix(y, y_kmeans))
print("\nClassification Report:")
print(classification_report(y, y_kmeans))

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

```
df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
df['Cluster'] = y_kmeans
df['True Label'] = y

plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='PC1', y='PC2', hue='Cluster', palette='Set1', s=100, edgecolor='black',
alpha=0.7)
plt.title('K-Means Clustering of Breast Cancer Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title="Cluster")
plt.show()

plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='PC1', y='PC2', hue='True Label', palette='coolwarm', s=100,
edgecolor='black', alpha=0.7)
plt.title('True Labels of Breast Cancer Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title="True Label")
plt.show()

plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='PC1', y='PC2', hue='Cluster', palette='Set1', s=100, edgecolor='black',
alpha=0.7)
centers = pca.transform(kmeans.cluster_centers_)
plt.scatter(centers[:, 0], centers[:, 1], s=200, c='red', marker='X', label='Centroids')
plt.title('K-Means Clustering with Centroids')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title="Cluster")
```

plt.show()

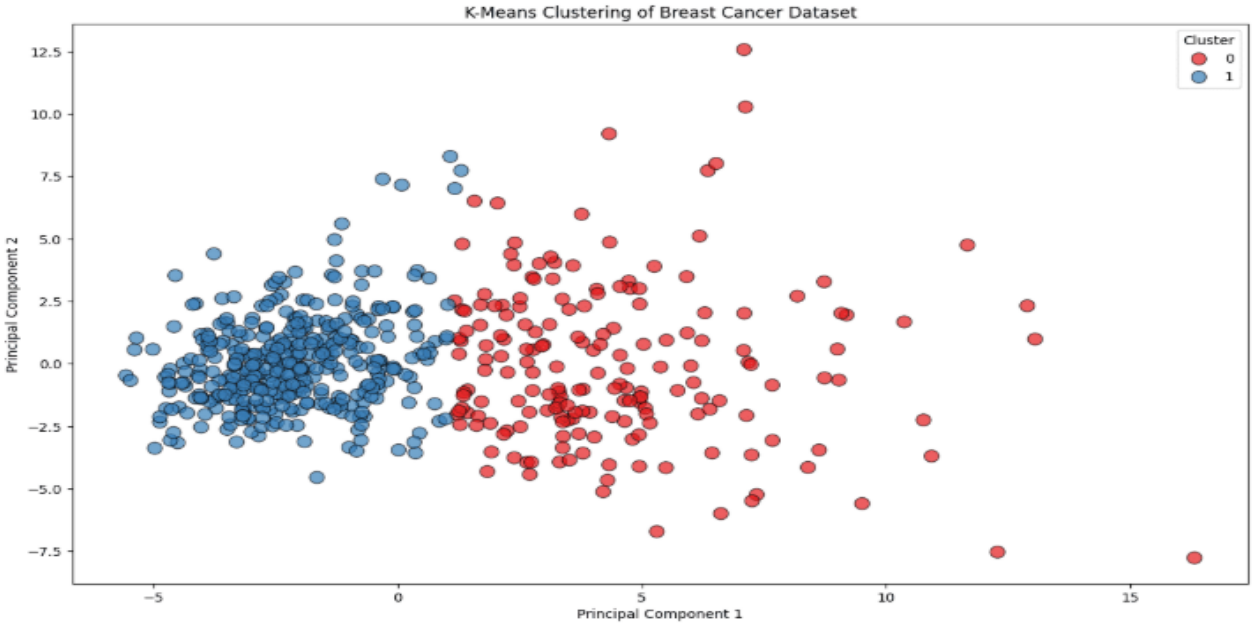
Output

```
Confusion Matrix:
[[175  37]
 [ 13 344]]

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.83      0.88       212
           1       0.90      0.96      0.93       357

    accuracy                           0.91       569
   macro avg       0.92      0.89      0.90       569
weighted avg       0.91      0.91      0.91       569
```



GM UNIVERSITY

(Established under the Karnataka State Act No. 19 of 2023)
Post Box no-4, PB Road, Davangere-577006

Faculty of Engineering and Technology

School of Computer Science and Technology

